AI Usage Report

Smart Task & Project Management System | 27 October 2026

Tools and scope

I used OpenAI Codex throughout planning and development, including the React interface, Express API, PostgreSQL schema, tests, Docker setup and deployment configuration. Standard starter files came from official generators. Most application code and tests were generated or edited with Codex's assistance.

Three actual prompts

The following English prompts are professionally edited summaries of my actual requests, not verbatim chat quotations:

1. Analyze the assignment and map every required and optional feature to acceptance criteria. Propose a maintainable React, Express and PostgreSQL architecture, a milestone plan, GitHub workflow and free hosting without payment-card requirements.
2. Clarify unresolved product and architecture decisions through concise multiple-choice questions. Record the agreed choices and obtain approval before implementation milestones, installations and publication.
3. Audit the implementation against the assignment and separation-of-concerns principles. Correct verified gaps, validate the application with database and browser tests, and prepare accurate setup, evaluation and AI-disclosure documents.

Problem solved with AI assistance

One useful example was the member permission rule: members can see shared work but can update only the status of their assigned tasks. Codex helped implement backend role checks and an atomic database update. API and browser tests checked that members could not edit other fields or update another person's task.

An incomplete suggestion and its correction

The first AI-assisted deployment configuration used npm ci under production NODE_ENV. That left out development dependencies needed to build the application. The install command was corrected to npm ci --include=dev, and CI was updated to check the same environment. This kept production runtime settings while making the build reproducible.

Verification and candidate contribution

Verification included formatting, lint, TypeScript checks, 43 automated tests, both application builds and ten desktop/mobile browser workflows. Integration tests used a real isolated PostgreSQL database. Container checks covered migrations and repeatable sample data; live checks covered authentication, permissions, persistence and the final Tasks interface. GitHub CI passed for the submitted application revision.

My contributions included choosing PostgreSQL, deciding permissions and deadline behavior, reviewing milestones, setting up local databases and hosting accounts, entering credentials privately and performing the initial deployment steps. I manually tested the application, including mobile use, and requested the final architecture and requirements review. Codex generated or edited the implementation and tests and ran the recorded automated checks. Detailed evidence is available in docs/AI-WORK-LOG.md and docs/FINAL-VERIFICATION.md.